



Faculty of Engineering and Technology
Department of Electrical and Computer Engineering
ENCS3320-Computer Networks
Project 1 Report: Socket Programming

Prepared by: Lara Abu Asfour 1221484

Hala Sabobeh 1220322

Tasneem Shelleh 1220439

Instructor: Dr. Ahmed Shawahna,
Dr. Ibrahim Nemer

Section: Lara Sec 1

Hala & Tasneem Sec 4

Date: 1/12/2024

Abstract

This project focuses on learning networking concepts through hands-on tasks with socket programming. We will build a simple web server using TCP sockets to serve dynamic, multilingual web pages and create a multiplayer trivia game using UDP sockets to explore real-time client-server communication. Additionally, the project includes using network commands like ping and tracert and analyzing packets with Wireshark. Through these activities, we aim to better understand networking protocols, develop practical programming skills, and improve teamwork.

Table of Contents

Abstract.....	II
Table of Figures	IV
Theory and Procedure	1
1. Task 1-Network Commands and Wireshark.....	1
2. Task 2-Web Server	1
3. Task 3-UDP Client-Server Trivia Game Using Socket Programming.....	2
Results and Discussions:	3
1. Task 1-Network Commands and Wireshark.....	3
2. Task 2-Web Server	10
3. Task 3-UDP Client-Server Trivia Game Using Socket Programming.....	20
Codes & Discussion:	20
• The server code:	20
• Server code discussion:	22
• The client code:	23
• Client code discussion:	23
Results and Discussion:	24
- The first example:	24
- The second example:	25
- More than one round example:.....	26
• Discussion:	27
Conclusion	28
Teamwork.....	29
References	30

Table of Figures

Figure 1: Run the ipconfig /all command.....	4
Figure 2: command for ping a device.....	5
Figure 3: Ping discover.engineering.utoronto.ca.....	5
Figure 4: IP details for 23.185.0.2	6
Figure 5: tracert discover.engineering.utoronto.ca.....	6
Figure 6: nslookup to retrieve the DNS information for discover.engineering.utoronto.ca.	7
Figure 7: telnet connection on port 23	7
Figure 8: telnet to discover.engineering.utoronto.ca on port 80.....	8
Figure 9: results of telnet to discover.engineering.utoronto.ca. on port 80.....	8
Figure 10: Some DNS messages from Wireshark.....	9
Figure 11: Terminal usage to run the web server program	10
Figure 12: Terminal shows how the request handled by server.....	10
Figure 13: Website	10
Figure 14: Website	11
Figure 15: Website	11
Figure 16: Website	11
Figure 17: supporting material web page.....	12
Figure 18: Result of searching for existing image (h.png).....	12
Figure 19: Screenshots of terminal showing how the requests handled when searching for existing image (h.png).....	12
Figure 20: searching for existing image (TCP.jpg).....	13
Figure 21: Result for searching for existing image (TCP.jpg)	13
Figure 22: searching for element that does not exist in our project	14
Figure 23: Error 404 page	14
Figure 24: Searching for non- existing image.....	15
Figure 25: Result of searching for non-existing image (http protocol.png).....	15
Figure 26: searching for non-existing video (http protocol.mp4).....	16
Figure 27: Result of searching for http protocol.mp4	16
Figure 28: Textbook website.....	17
Figure 29: Screenshots of terminal showing how the requests handled when enter to textbook website	17
Figure 30: Ritaj Website.....	18
Figure 31: Arabic Website.....	18
Figure 32: Arabic Website.....	19
Figure 33: Arabic supporting material	19
Figure 34:result of searching for existing image (lara.png	19
Figure 35: server code.....	20
Figure 36: server code.....	20
Figure 37: server code.....	20
Figure 38: server code.....	21
Figure 39: server code.....	21
Figure 40: server code.....	21
Figure 41: the client code	23

Figure 42: the client code	23
Figure 43: the output of server Ex1	24
Figure 44: the output of client1 Ex1	24
Figure 45: the output of client2 EX1	24
Figure 46: output of server EX2	25
Figure 47: output of client1 EX2	25
Figure 48: output of client2 EX2	25
Figure 49: output of server EX3	26
Figure 50: output of client1 EX3	26
Figure 51: output of client2 EX3	26
Figure 52: Work distribution	29

Theory and Procedure

1. Task 1-Network Commands and Wireshark

Network commands such as `ipconfig`, `ping`, `tracert`, `telnet`, and `nslookup` are essential tools for diagnosing and analyzing network behavior. These commands provide valuable insights into various aspects of network communication. For instance, `ipconfig` displays the system's IP address, subnet mask, and gateway, allowing users to identify network configuration issues. The `ping` command tests connectivity to a host by sending ICMP echo requests and measuring response times, while `tracert` reveals the route packets take to a destination, identifying delays or failures at specific hops. `Telnet` facilitates remote communication with servers, enabling manual interactions with services on specific ports, and `nslookup` queries DNS servers to resolve domain names into IP addresses or vice versa. Wireshark, a network packet analyzer, complements these tools by capturing and analyzing network traffic in real time, allowing users to inspect protocols like DNS, HTTP, and TCP/IP. Together, these tools are invaluable for understanding network functionality and troubleshooting problems.

In this task, we use these components:

- The terminal is where we run network commands like `ping` and `tracert` to check connections and understand how data moves through the network.
- Wireshark, on the other hand, acts like a microscope for network traffic, letting us capture and analyze the data packets to see exactly what's happening behind the scenes.

2. Task 2-Web Server

A web server is like a digital librarian it listens for requests from users and delivers the information or files they ask for, such as webpages, images, or videos. In this project, the web server is built using socket programming, which allows it to communicate with users through a network. It listens on a specific port, handles client requests, and responds with content like HTML pages styled with CSS to make them look nice and professional. If a user asks for something the server doesn't have, it uses a redirection status, like 307 Temporary Redirect, to guide them to another resource. This task demonstrates how web servers work behind the scenes to provide the information we access every day, using a mix of programming and web development techniques to create a seamless experience.

In this task, we use these components:

- Socket library: This is like the communication bridge between the server and the client, making sure they can send and receive messages over the network.
- HTML/CSS and Python files: Think of these as the building blocks of a webpage HTML organizes the content, while CSS makes it look good and easy to read on the screen, Python was used for the server and client codes due to its simplicity and readability.
- Redirection logic: When something isn't found, this part steps in to send the user to the right place, like a helpful guide directing you to the correct page if the original one isn't available.
- The terminal is used to run the web server program, display incoming HTTP request details, and provide debugging or status messages about server responses.

3. Task 3-UDP Client-Server Trivia Game Using Socket Programming

Imagine playing a trivia game where multiple players connect to a server, answer questions, and compete for the top score all in real time. This is made possible using UDP, a fast and lightweight protocol designed for quick communication. Unlike other protocols like TCP, which ensures every packet of data arrives safely, UDP focuses on speed, making it perfect for games where every second counts. The server acts as the game host, sending questions to all players and keeping track of their answers and scores. Players, as clients, receive the questions, submit their responses, and wait for updates from the server. This task shows how UDP and socket programming come together to create a fun and interactive multiplayer experience, while balancing speed and reliability in a distributed environment.

In this task, we use these components:

- Python was used for the server and client codes due to its simplicity and readability
- the terminal was like our game's dashboard. It showed us when players joined, displayed the trivia questions, tracked their answers, and updated the scores, letting us see everything happening in real time as the game progressed.

Results and Discussions:

1. Task 1-Network Commands and Wireshark

The explanation of each of the following commands:

Ipconfig: This command shows our computer's network details, like its IP address and connection info. It's handy for fixing network issues or updating your IP settings.

Ping: This command checks network connectivity by sending small data packets between devices and measuring the round-trip time (RTT).

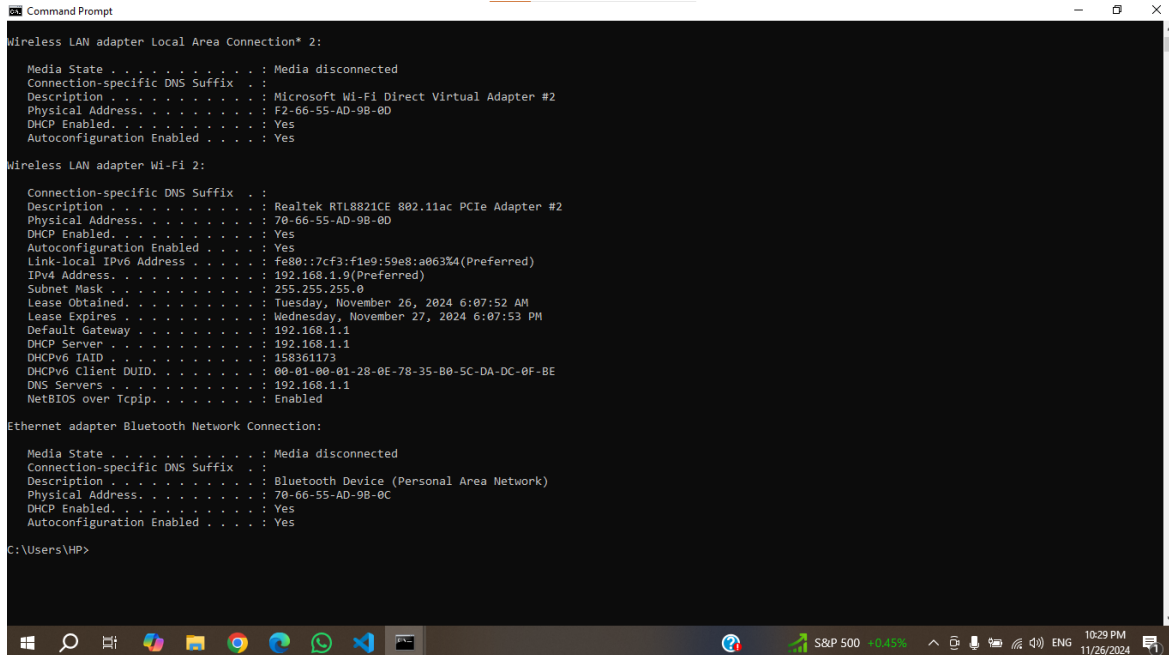
Tracert: Used to trace the route taken by data through the network, this command displays each hop (such as routers) and the latency at each step.

Telnet: A protocol that enables a command-line connection to a remote device over the network, allowing direct communication and control.

Nslookup: A tool for querying DNS to find the IP address associated with a hostname or performing a reverse lookup to determine the hostname for a given IP address.

After we ensure that our computer is connected to the internet, then perform the following actions:

The **ipconfig /all** command run on our computer, and the IP address, subnet mask, default gateway, and Domain Name System (DNS) server addresses for our primary network interface identified.



```
Command Prompt

Wireless LAN adapter Local Area Connection* 2:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : 
    Description . . . . . : Microsoft Wi-Fi Direct Virtual Adapter #2
    Physical Address. . . . . : F2-66-55-AD-9B-0D
    DHCP Enabled. . . . . : Yes
    Autoconfiguration Enabled . . . . : Yes

Wireless LAN adapter Wi-Fi 2:

    Connection-specific DNS Suffix  . : 
    Description . . . . . : Realtek RTL8821CE 802.11ac PCIe Adapter #2
    Physical Address. . . . . : 70-66-55-AD-9B-0D
    DHCP Enabled. . . . . : Yes
    Autoconfiguration Enabled . . . . : Yes
    Link-local IPv6 Address . . . . . : fe80::7cf3:f1e9:59e8:a063%4(Preferred)
    IPv4 Address. . . . . : 192.168.1.9(Preferred)
    Subnet Mask . . . . . : 255.255.255.0
    Lease Obtained. . . . . : Tuesday, November 26, 2024 6:07:52 AM
    Lease Expires . . . . . : Wednesday, November 27, 2024 6:07:53 PM
    Default Gateway . . . . . : 192.168.1.1
    DHCP Server . . . . . : 192.168.1.1
    DHCPv6 IAID . . . . . : 158361173
    DHCPv6 Client DUID. . . . . : 00-01-00-01-28-0E-78-35-B0-5C-DA-DC-0F-BE
    DNS Servers . . . . . : 192.168.1.1
    NetBIOS over Tcpip. . . . . : Enabled

Ethernet adapter Bluetooth Network Connection:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : 
    Description . . . . . : Bluetooth Device (Personal Area Network)
    Physical Address. . . . . : 70-66-55-AD-9B-0C
    DHCP Enabled. . . . . : Yes
    Autoconfiguration Enabled . . . . : Yes

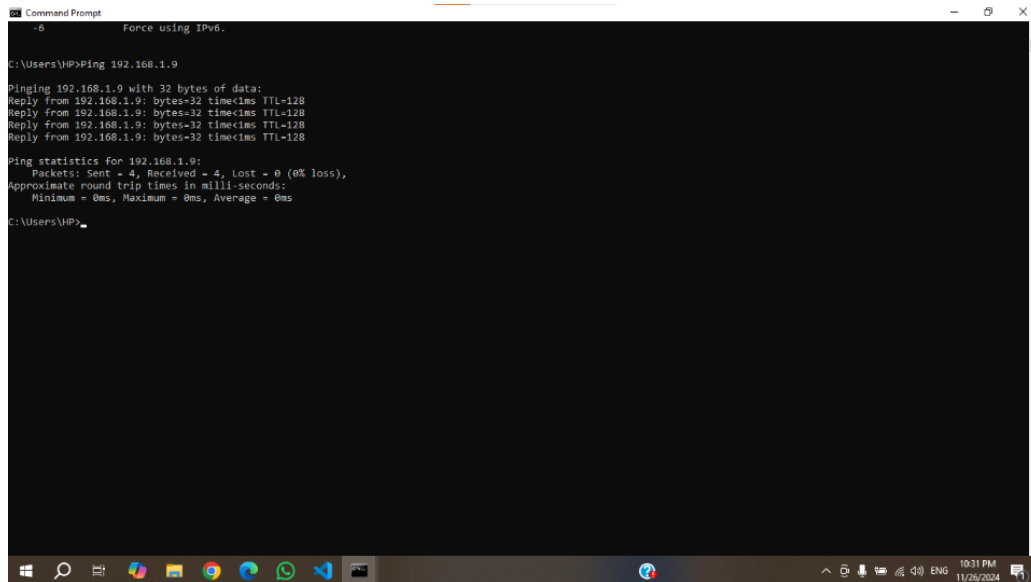
C:\Users\HP>
```

Figure 1: Run the ipconfig /all command.

As shown in Figure 1:

- The IP address is 192.168.1.9
- The subnet mask is 255.255.255.0
- The default gateway is 192.168.1.1
- The DNS server is 192.168.1.1

We Ping a device within our local network (e.g., from our laptop to a smartphone on the same Wi-Fi network).



```
Command Prompt
Force using IPv6.

C:\Users\HP>Ping 192.168.1.9

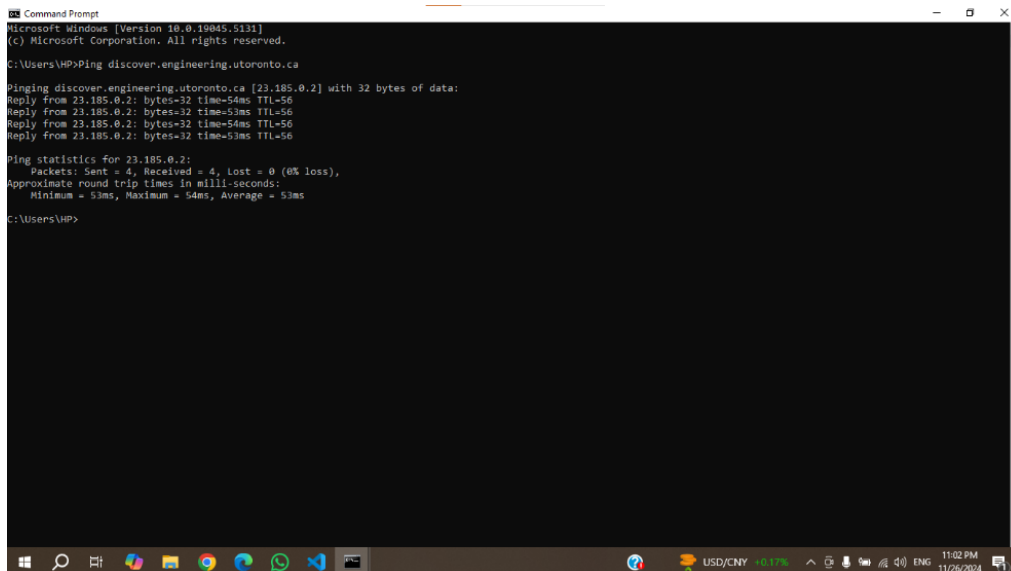
Pinging 192.168.1.9 with 32 bytes of data:
Reply from 192.168.1.9: bytes=32 time=0ms TTL=128
Reply from 192.168.1.9: bytes=32 time=0ms TTL=128
Reply from 192.168.1.9: bytes=32 time=0ms TTL=128
Reply from 192.168.1.9: bytes=32 time=0ms TTL=128

Ping statistics for 192.168.1.9:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\Users\HP>
```

Figure 2: command for ping a device.

We Ping discover.engineering.utoronto.ca



```
Command Prompt
Microsoft Windows [Version 10.0.19045.5131]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP>Ping discover.engineering.utoronto.ca

Pinging discover.engineering.utoronto.ca [23.185.0.2] with 32 bytes of data:
Reply from 23.185.0.2: bytes=32 time=54ms TTL=56
Reply from 23.185.0.2: bytes=32 time=53ms TTL=56
Reply from 23.185.0.2: bytes=32 time=54ms TTL=56
Reply from 23.185.0.2: bytes=32 time=53ms TTL=56

Ping statistics for 23.185.0.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 53ms, Maximum = 54ms, Average = 53ms

C:\Users\HP>
```

Figure 3: Ping discover.engineering.utoronto.ca.

Based on the results, we believe that the response does not originate from Canada, that when we ping discover.engineering.utoronto.ca, based on its IP address 23.185.0.2 the response showed that the server is actually located in California, even though the name suggests it's tied to the University of Toronto in Canada. the server's physical location might not always match what the domain name implies. In this case, even though the domain points to Canada, the server itself is in California.

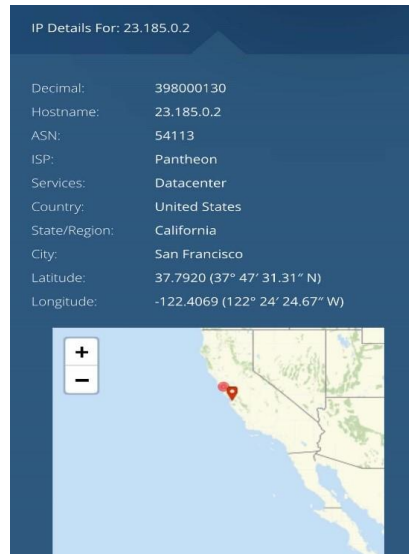


Figure 4: IP details for 23.185.0.2

We Run tracert on discover.engineering.utoronto.ca.

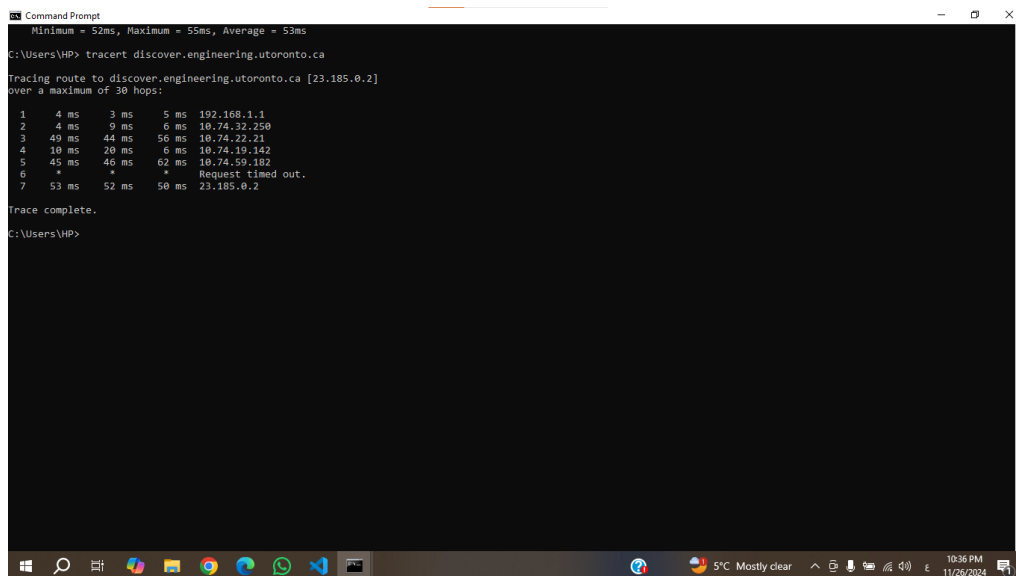
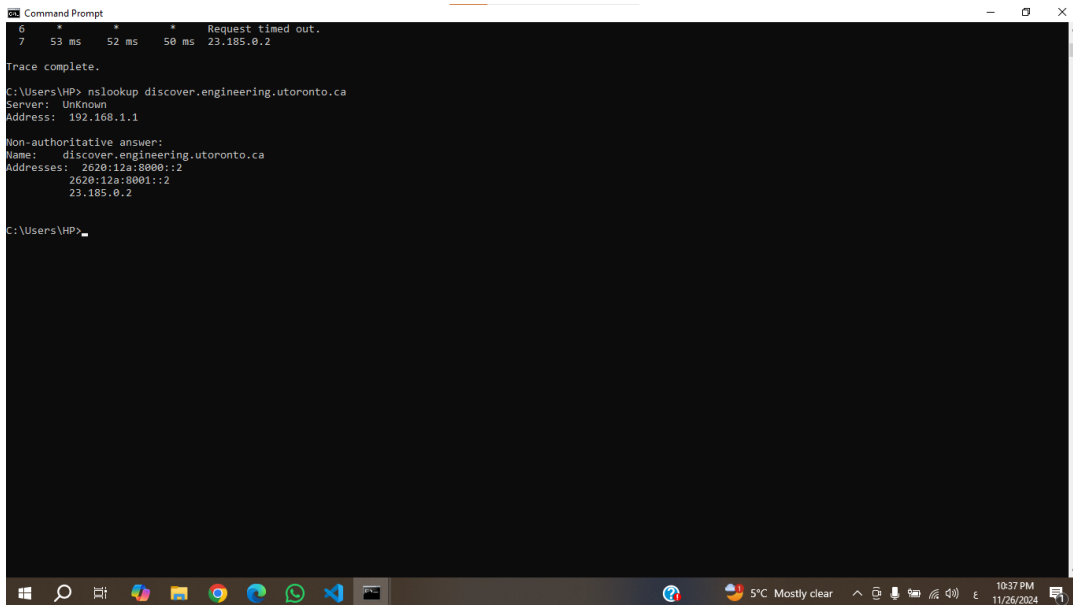


Figure 5: tracert discover.engineering.utoronto.ca.

We use nslookup to retrieve the DNS information for discover.engineering.utoronto.ca.



```
6 * * * Request timed out.
7 53 ms 52 ms 50 ms 23.185.0.2

Trace complete.

C:\Users\HP> nslookup discover.engineering.utoronto.ca
Server: Unknown
Address: 192.168.1.1

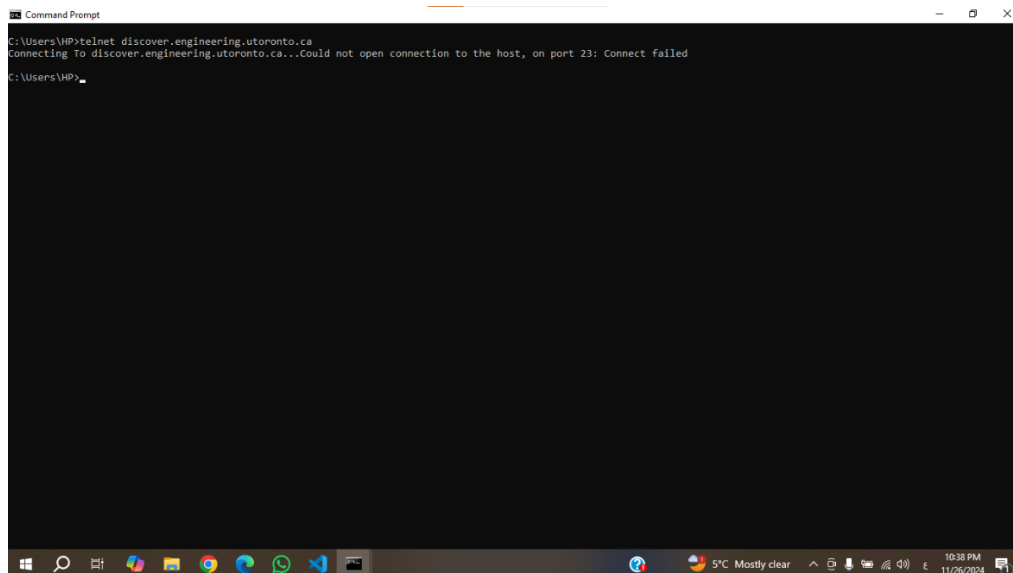
Non-authoritative answer:
Name:    discover.engineering.utoronto.ca
Addresses: 2620:12a:8000::2
          2620:12a:8001::2
          23.185.0.2

C:\Users\HP>
```

Figure 6: nsLookup to retrieve the DNS information for discover.engineering.utoronto.ca.

We tried to connect with telnet to discover.engineering.utoronto.ca. on two different ports, the results appear as the following:

As Figure 7 shown, telnet connection on port 23 (default port) was unsuccessful.



```
C:\Users\HP>telnet discover.engineering.utoronto.ca
Connecting To discover.engineering.utoronto.ca...Could not open connection to the host, on port 23: Connect failed

C:\Users\HP>
```

Figure 7: telnet connection on port 23

As Figure 8 shown, telnet connection on port 80 was successful and to interact with the server we send an HTTP request `GET/HTTP1.1 Host: discover.engineering.utoronto.ca`, then the response is shown in Figure 9.

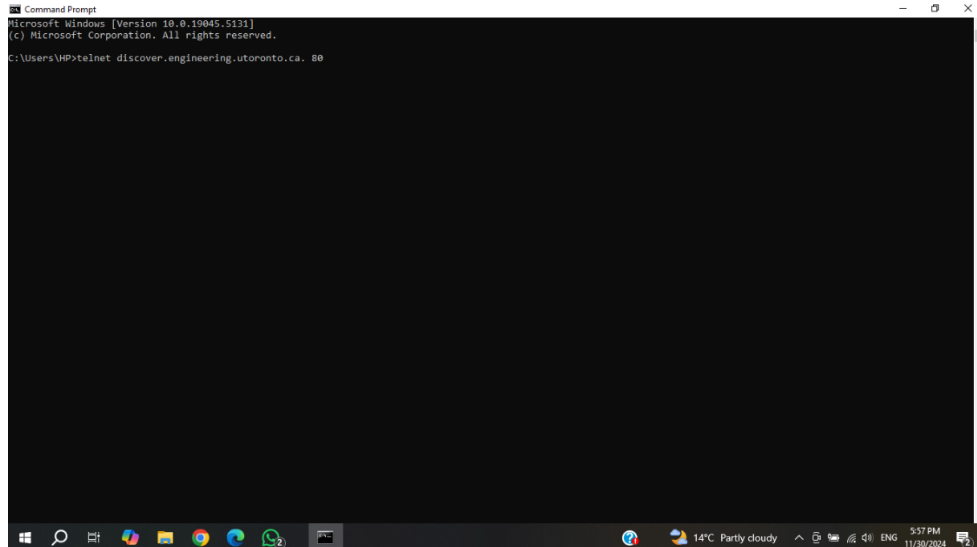


Figure 8: telnet to discover.engineering.utoronto.ca on port 80.

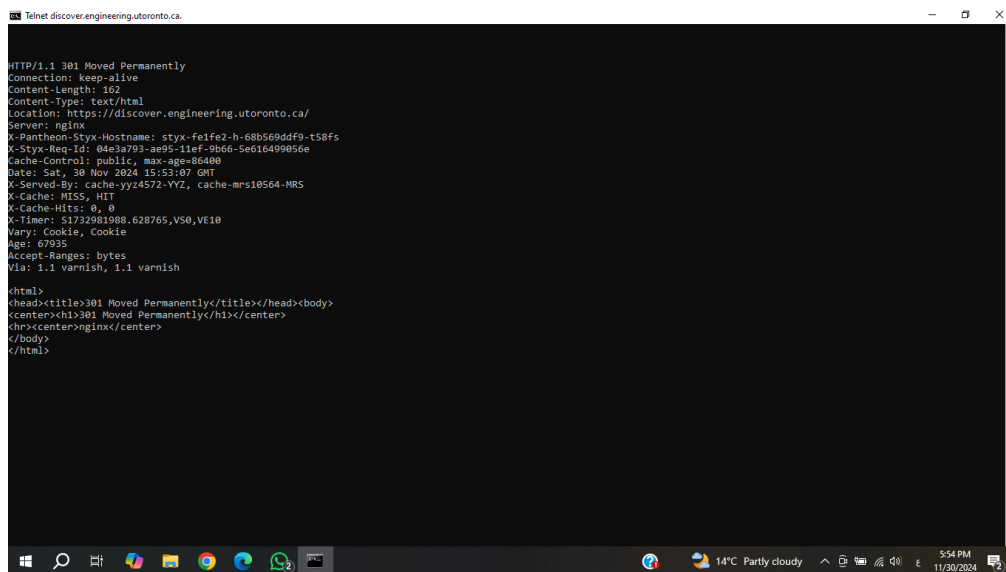


Figure 9: results of telnet to discover.engineering.utoronto.ca. on port 80

We use the Wireshark packet analyzer to capture a DNS query and reply for any hostname of your choice.

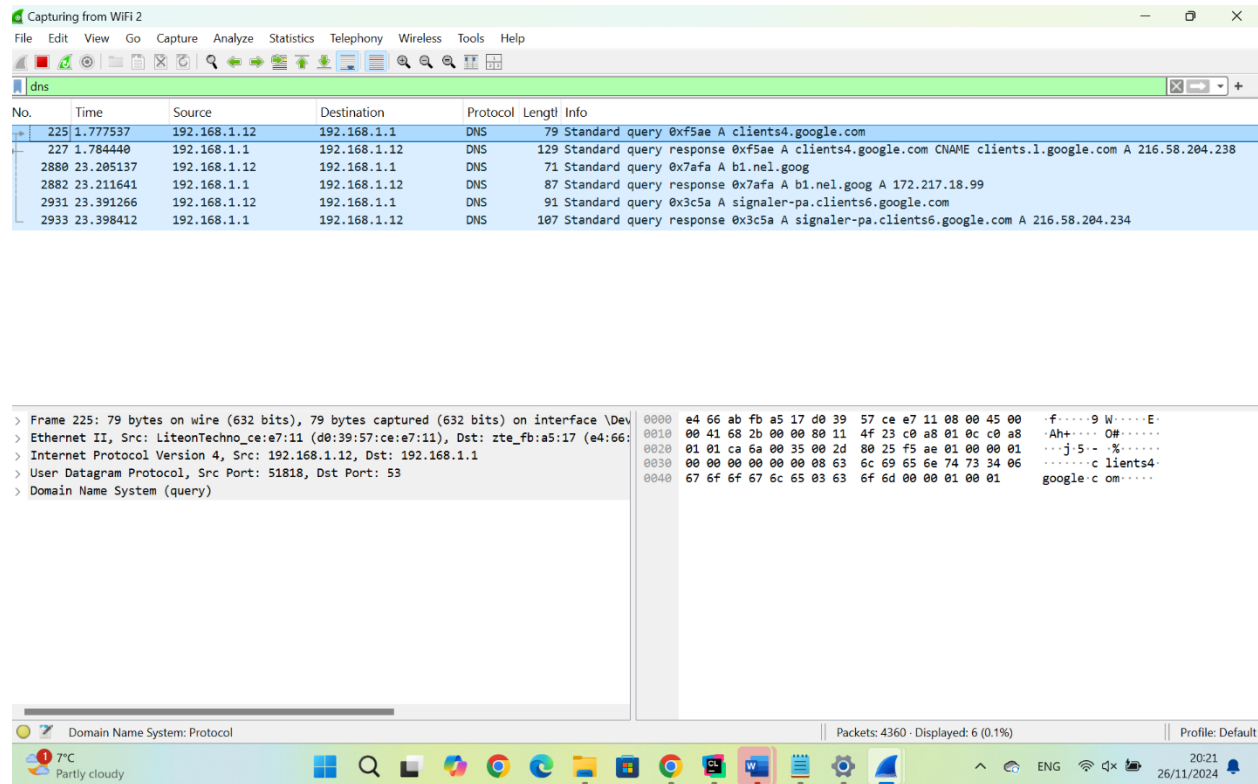


Figure 10: Some DNS messages from Wireshark.

2. Task 2-Web Server

The terminal is used to run the web server program.

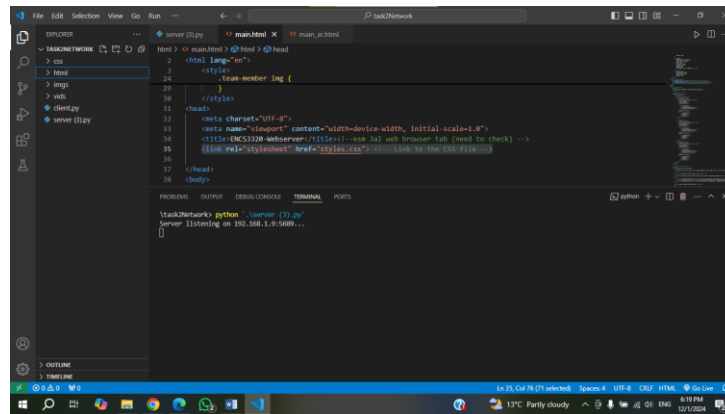


Figure 11: Terminal usage to run the web server program

Screenshots of terminal showing how the requests handled by server and then starting to present the website:

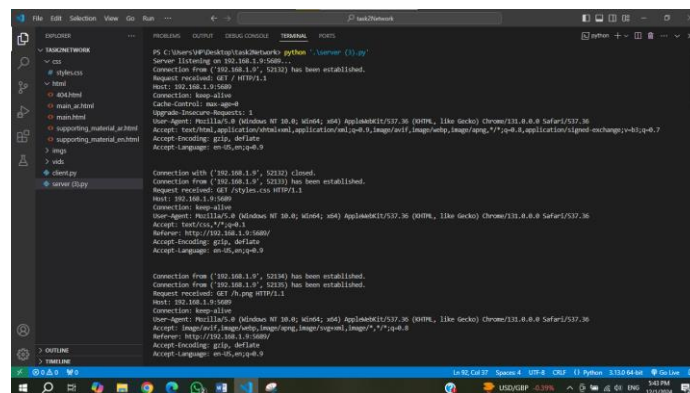


Figure 12: Terminal shows how the request handled by server

The website:

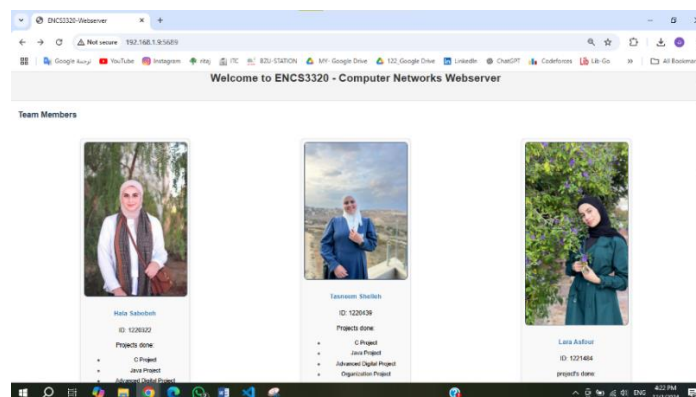


Figure 13: Website

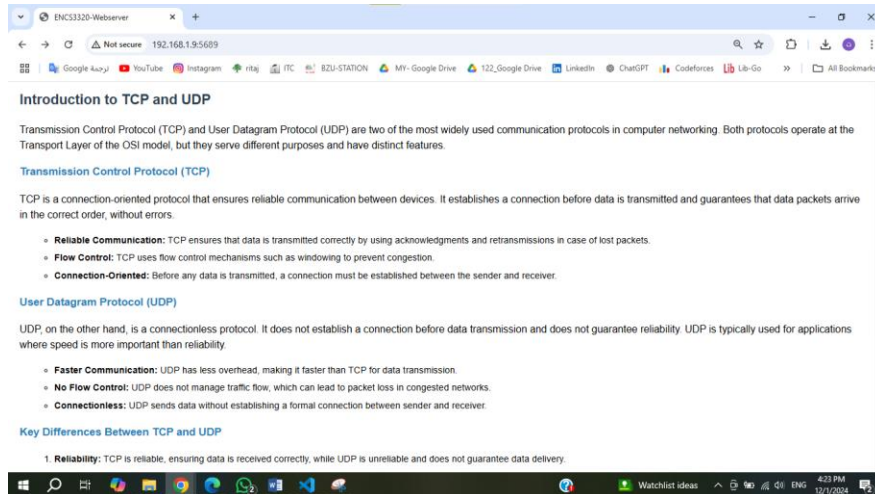


Figure 14: Website

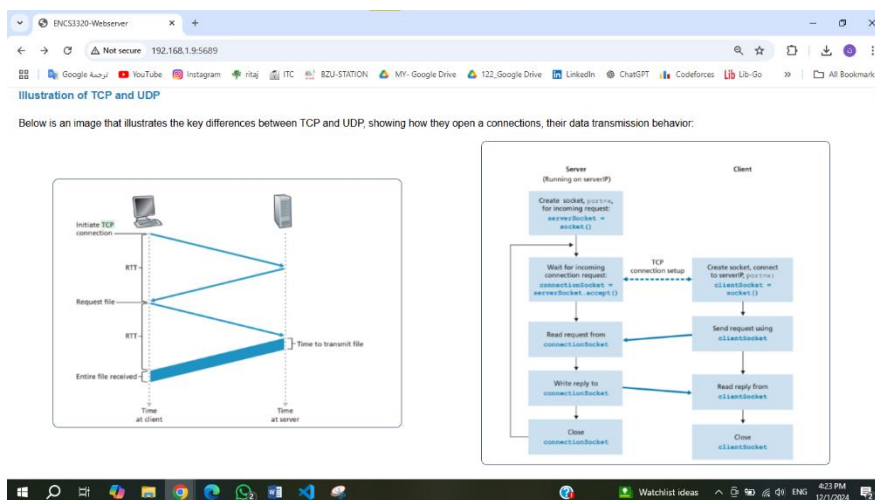


Figure 15: Website

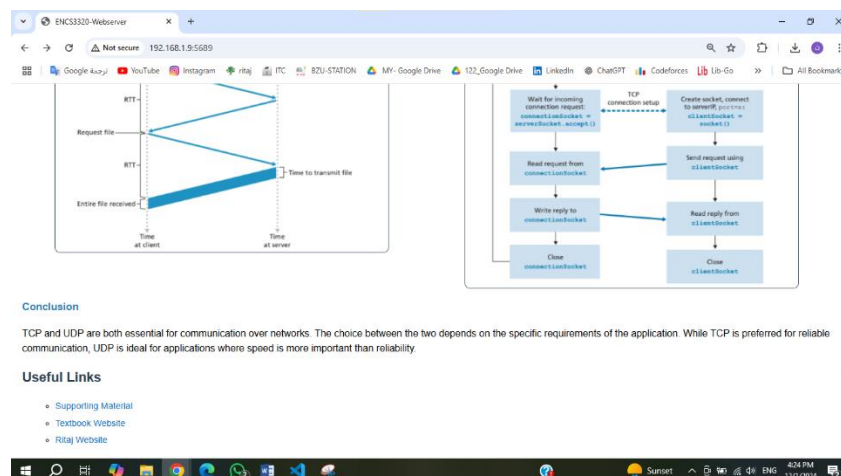


Figure 16: Website

Supporting material web page: searching for existing image (h.png)

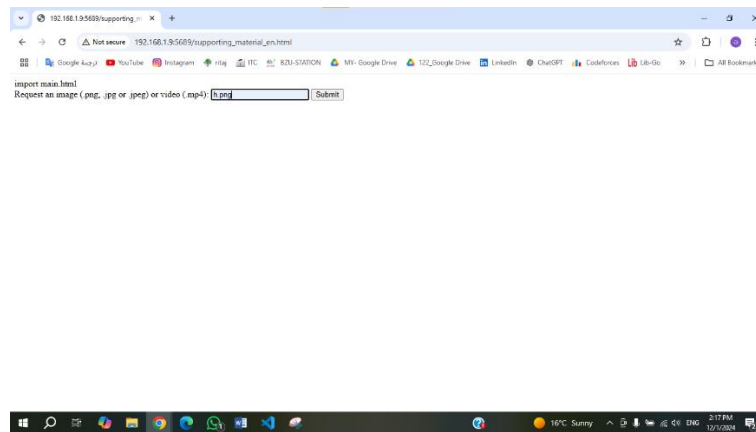


Figure 17: supporting material web page

Result of searching Supporting material web page: searching for existing image (h.png):

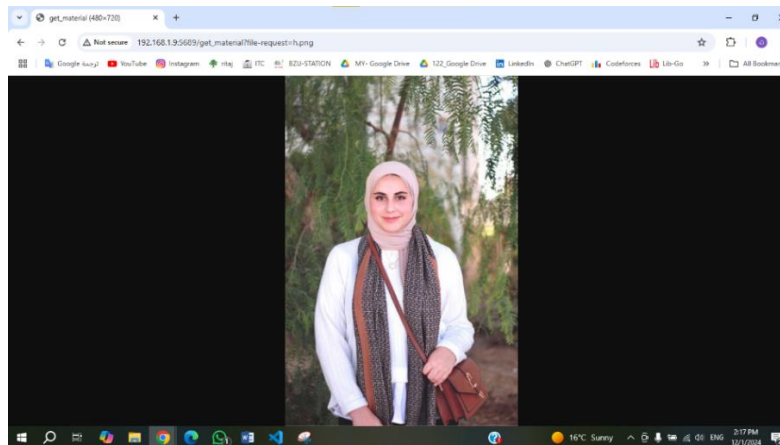


Figure 18: Result of searching for existing image (h.png)

Screenshots of terminal showing how the requests handled by server:

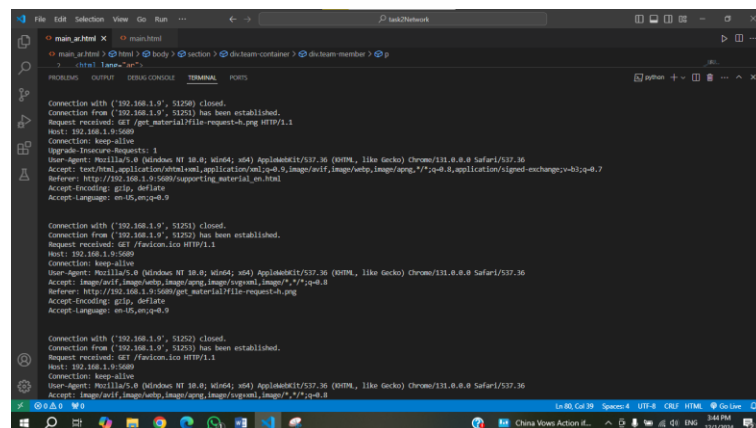


Figure 19: Screenshots of terminal showing how the requests handled when searching for existing image (h.png)

Supporting material web page: searching for existing image (TCP.jpg)

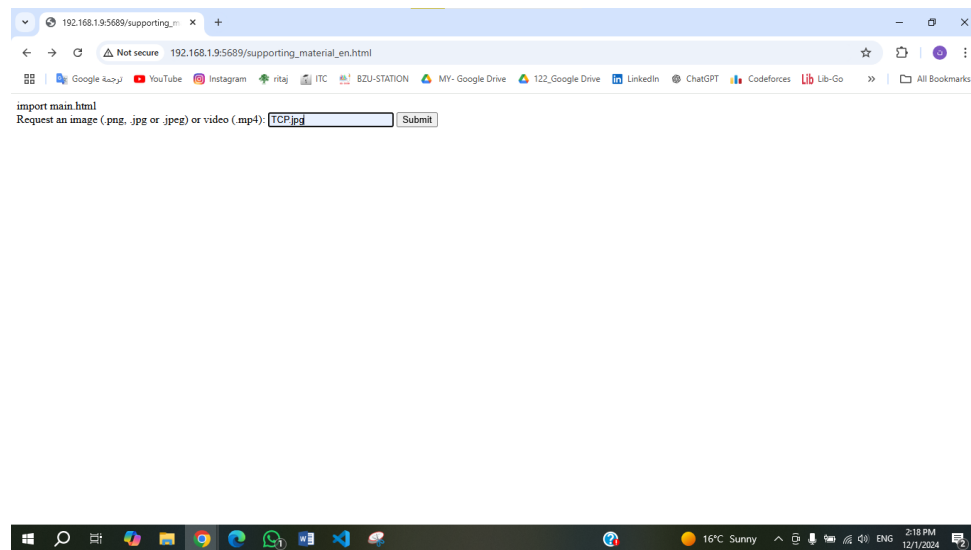


Figure 20: searching for existing image (TCP.jpg)

Result of searching Supporting material web page: searching for existing image (TCP.jpg):

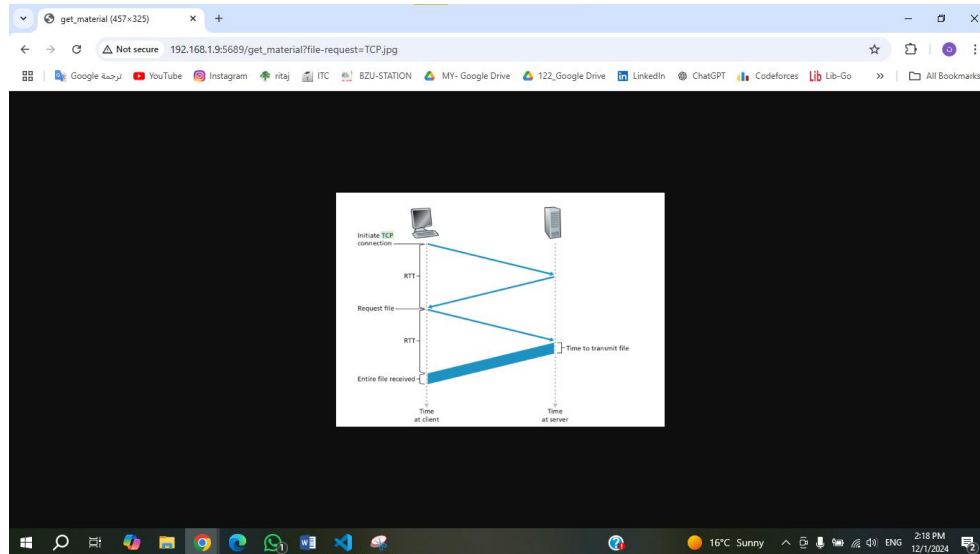


Figure 21: Result for searching for existing image (TCP.jpg)

Supporting material web page: searching for element that does not exist in our project

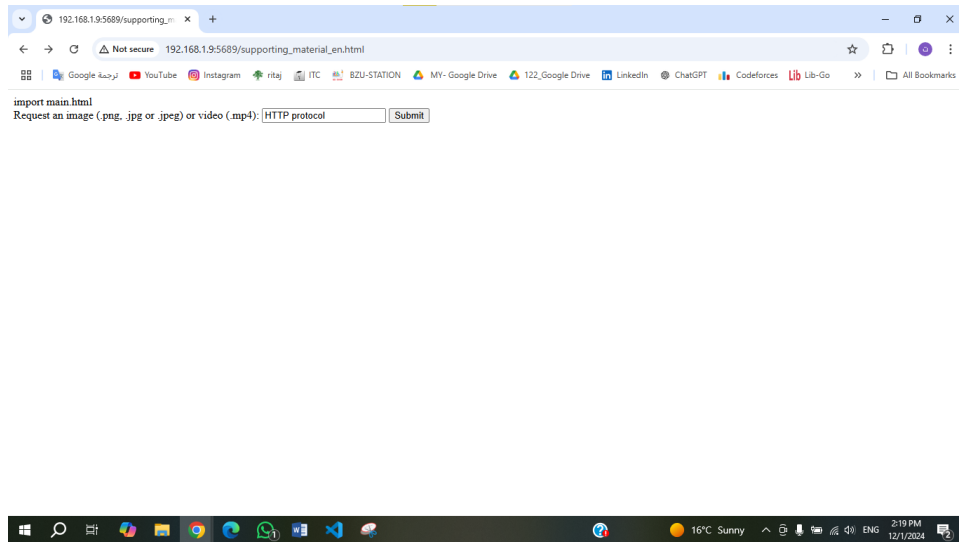


Figure 22: searching for element that does not exist in our project

Result of searching Supporting material web page: searching for: it gives us the Error 404 page because it is not an existing path or file

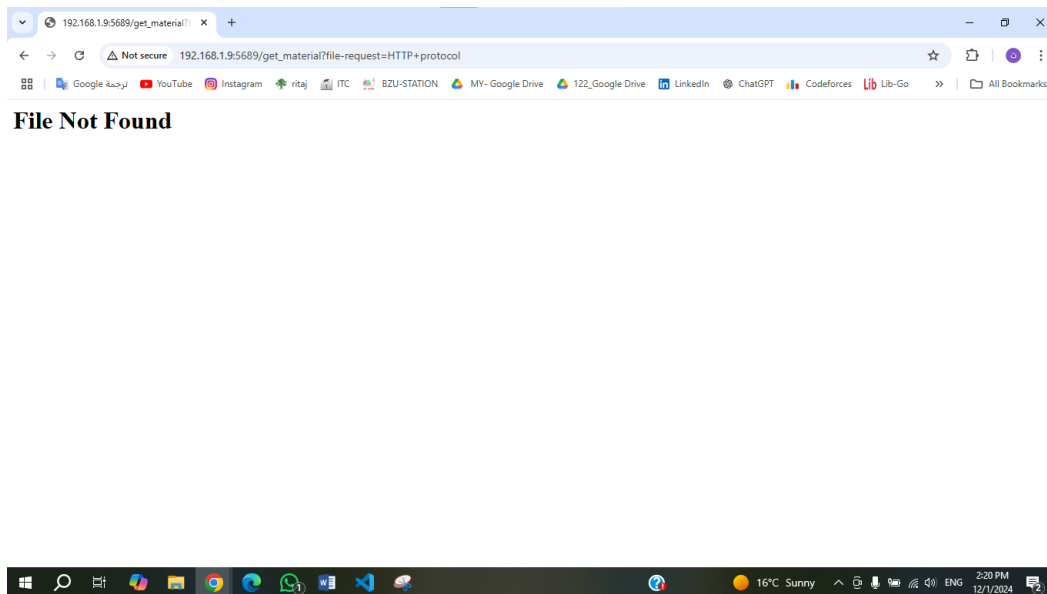


Figure 23: Error 404 page

Supporting material web page: searching for non-existing image (http protocol.png):

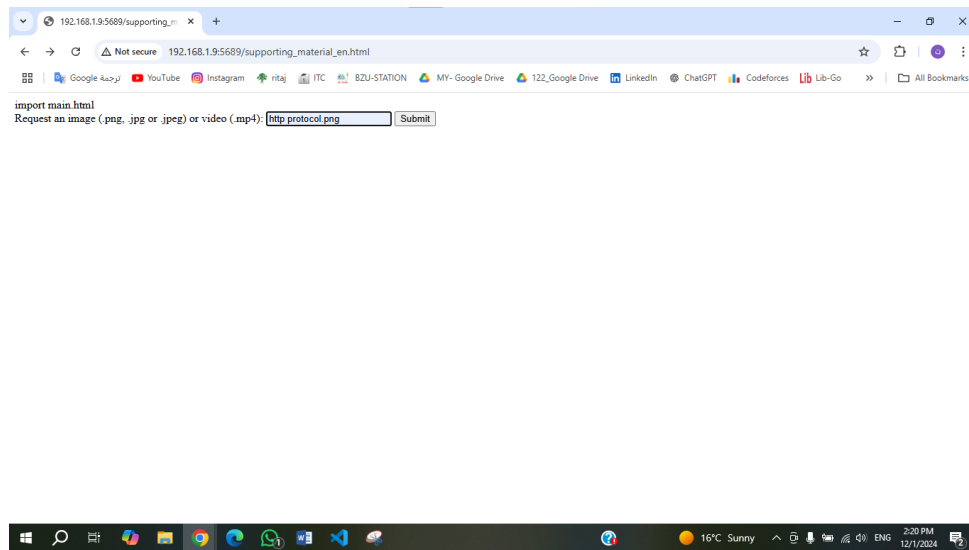


Figure 24: Searching for non-existing image

Result of searching Supporting material web page: searching for non-existing image (http protocol.png):

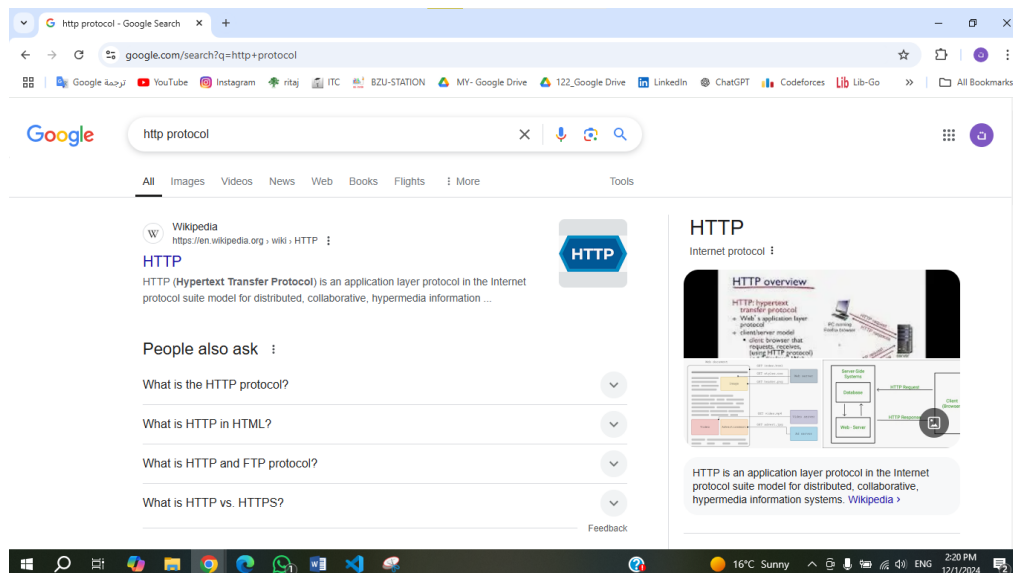


Figure 25: Result of searching for non-existing image (http protocol.png)

Supporting material web page: searching for non-existing video (http protocol.mp4):

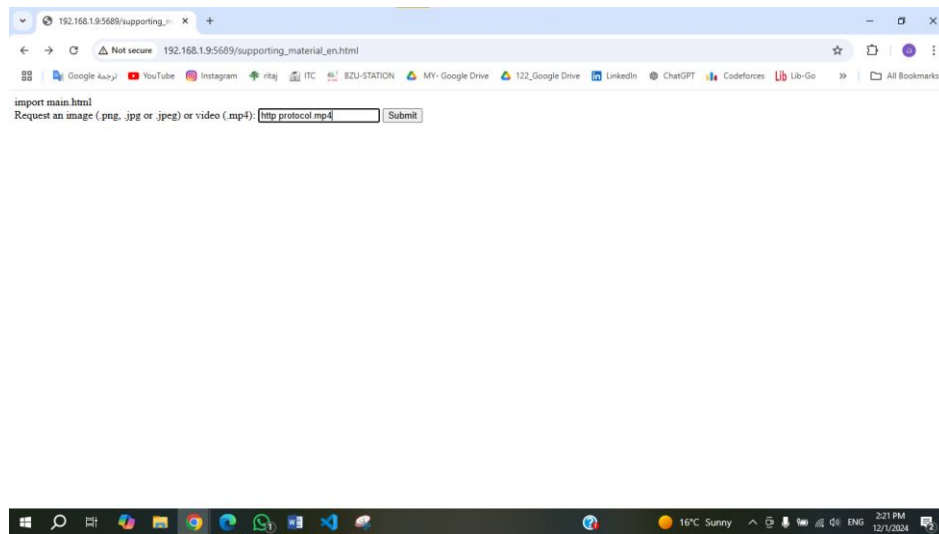


Figure 26: searching for non-existing video (http protocol.mp4)

Result of searching Supporting material web page: searching for non-existing video (http protocol.mp4):

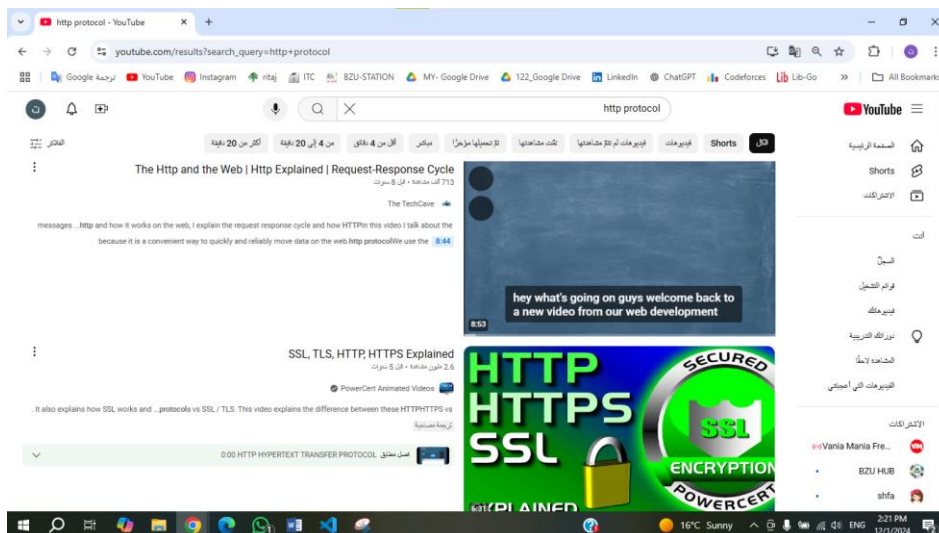


Figure 27: Result of searching for http protocol.mp4

Textbook Website: we can reach it from the button in the main.html page

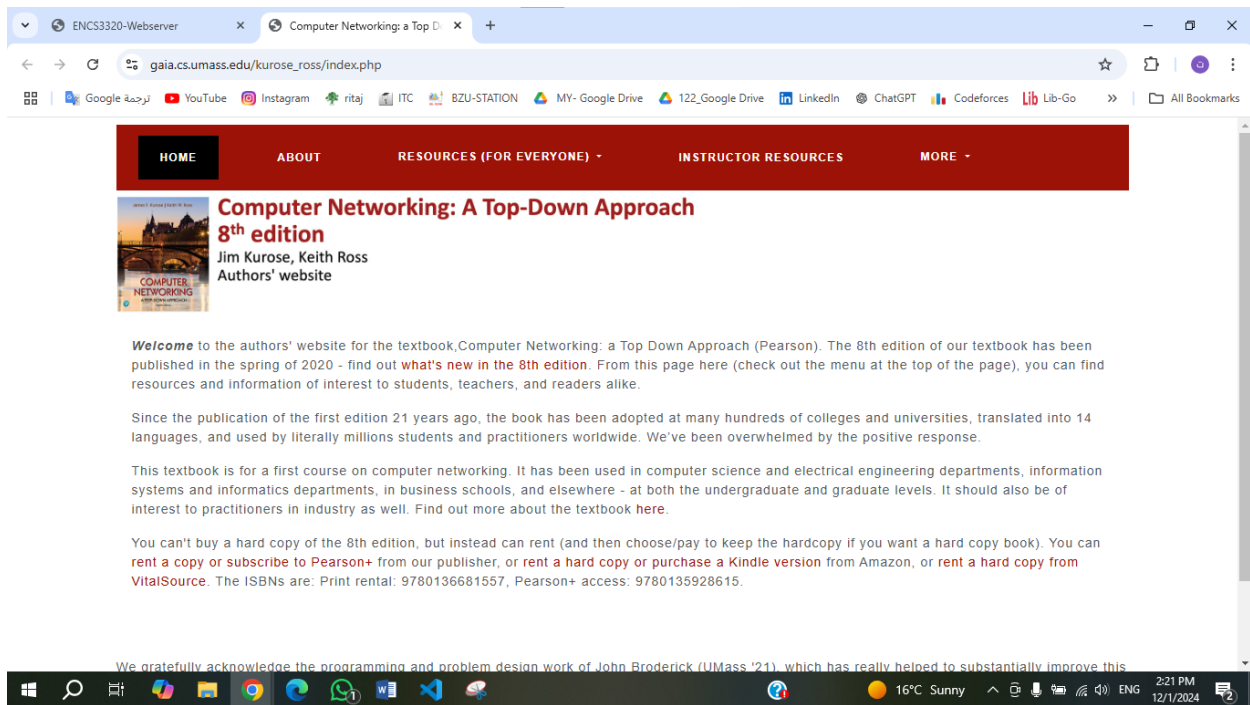


Figure 28: Textbook website

Screenshots of terminal showing how the requests handled by server when enter to textbook website:

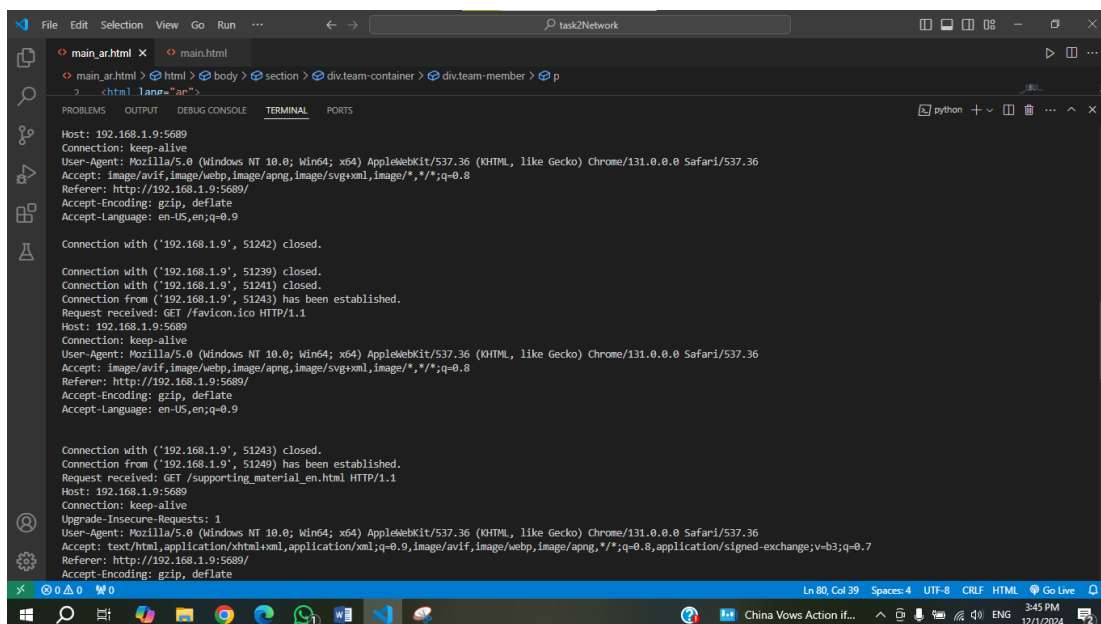


Figure 29: Screenshots of terminal showing how the requests handled when enter to textbook website

Ritaj Website: we can reach it from the button in the main.html page:

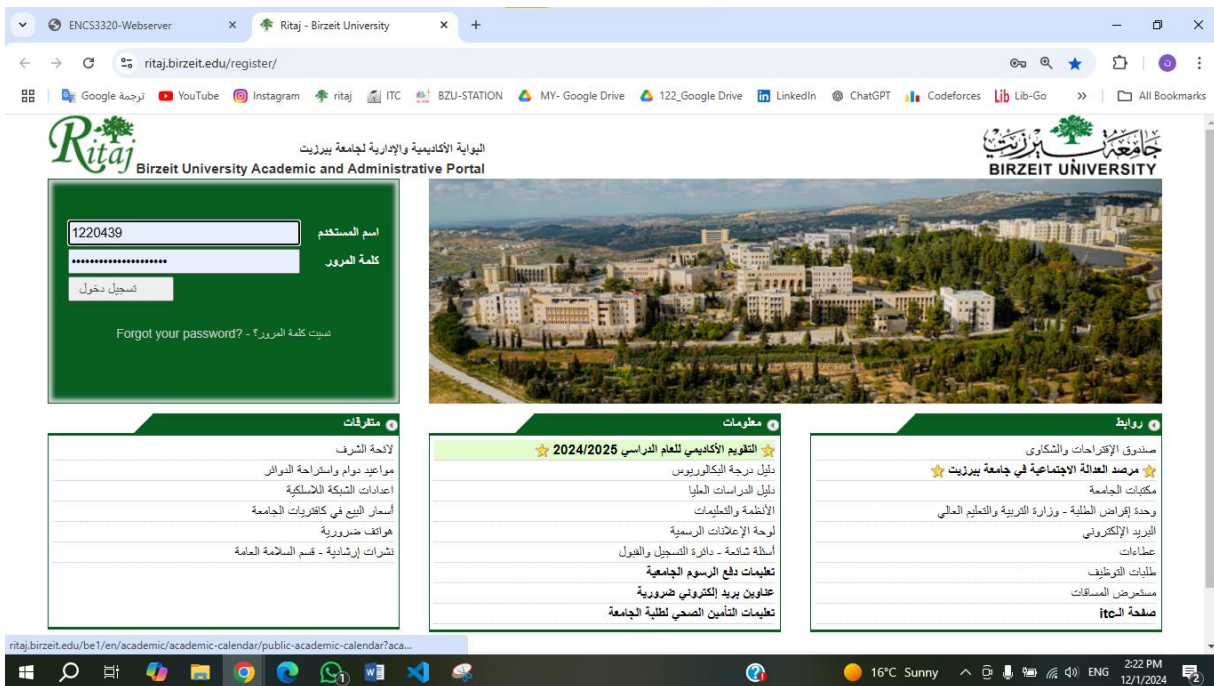


Figure 30: Ritaj Website

The Arabic Website that can reach by http://192.168.1.9:5689/main_ar.html :

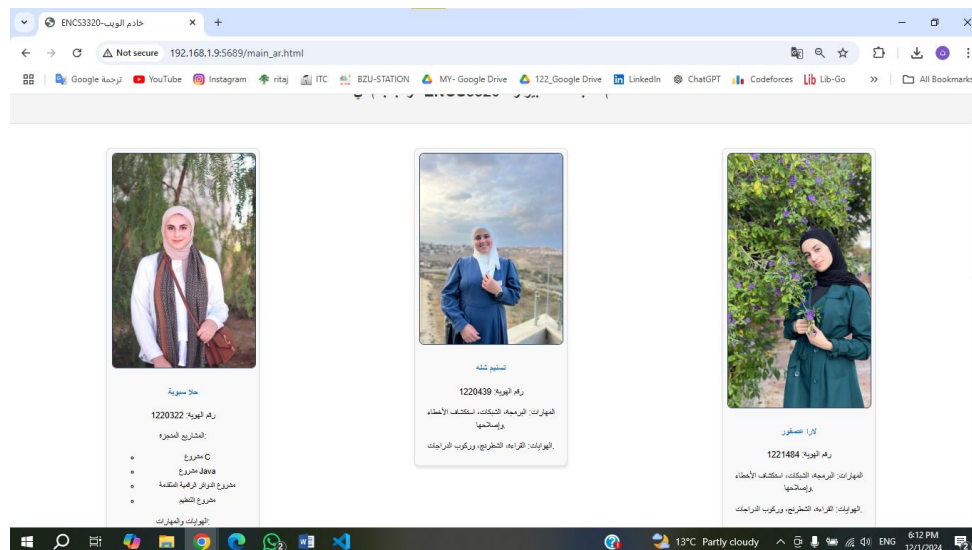


Figure 31: Arabic Website

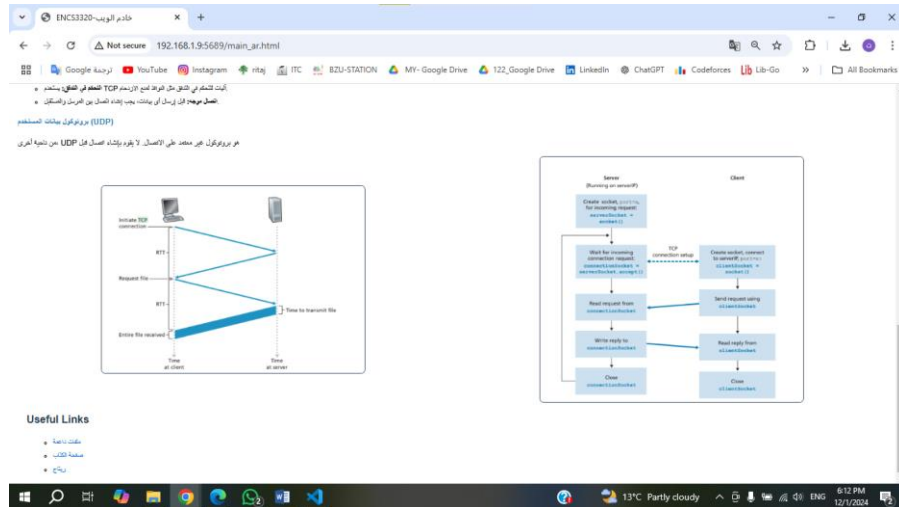


Figure 32: Arabic Website

Arabic supporting material:

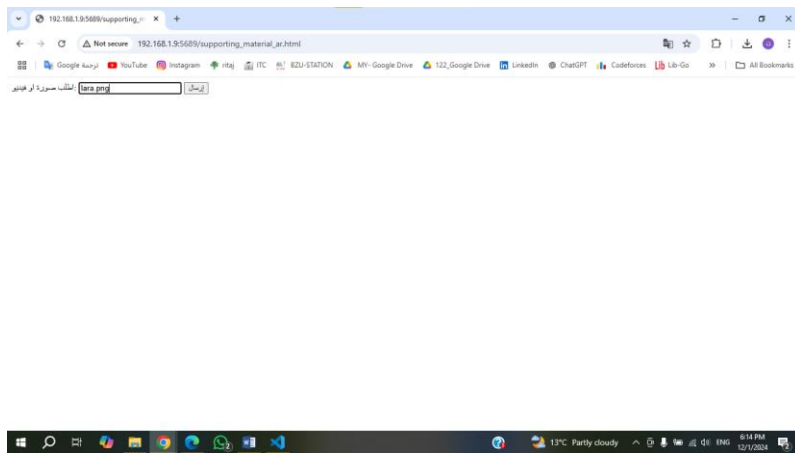


Figure 33: Arabic supporting material

The result Arabic of supporting material: searching for existing image (lara.png):

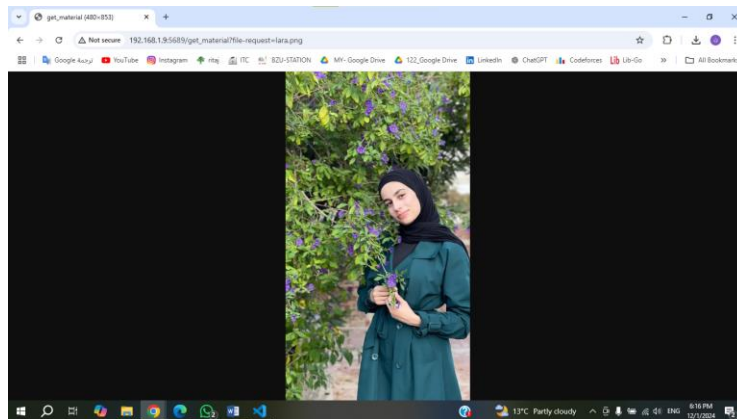
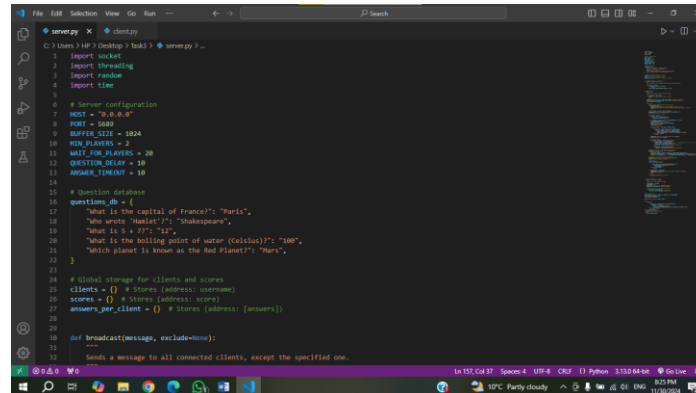


Figure 34: result of searching for existing image (lara.png)

3. Task 3-UDP Client-Server Trivia Game Using Socket Programming

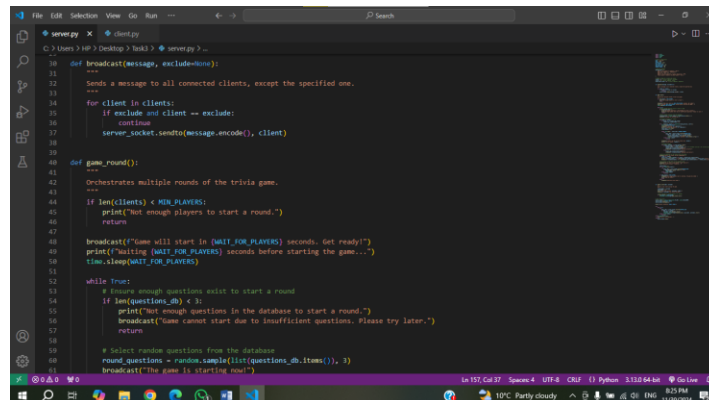
Codes & Discussion:

- The server code:



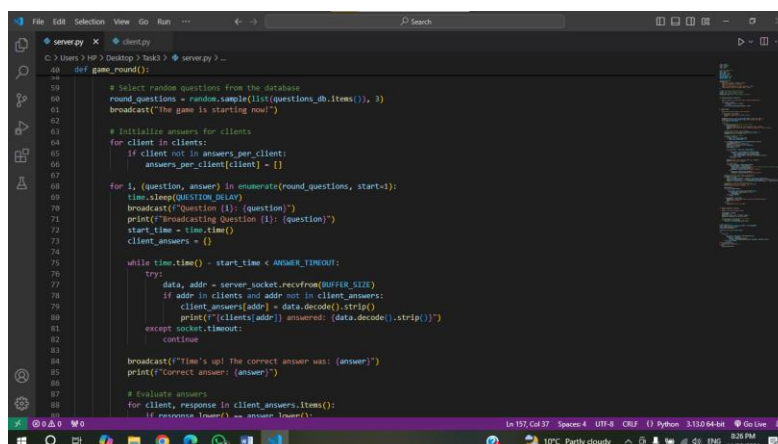
```
1 import socket
2 import threading
3 import random
4 import time
5
6 # Server configuration
7 HOST = "0.0.0.0"
8 PORT = 5438
9 BUFFER_SIZE = 1024
10 MAX_PLAYERS = 2
11 WAIT_FOR_PLAYERS = 20
12 QUESTION_DELAY = 10
13 ANSWER_TIMEOUT = 10
14
15 # Question database
16 question_db = [
17     "What is the capital of France?", "Paris",
18     "Who wrote 'Hamlet'?", "Shakespeare",
19     "What is 2 + 2?", "4",
20     "What is the boiling point of water (Celsius)?", "100",
21     "Which planet is known as the Red Planet?", "Mars",
22 ]
23
24 # Global storage for clients and scores
25 clients = {} # Stores (address, username)
26 scores = {} # Stores (address, score)
27 answers_per_client = {} # Stores (address, [answers])
28
29 def broadcast(message, exclude=None):
30     """
31     Sends a message to all connected clients, except the specified one.
32     """
```

Figure 35: server code



```
33     for client in clients:
34         if exclude and client == exclude:
35             continue
36         server_socket.sendto(message.encode(), client)
37
38 def game_round():
39     """
40     Orchestrates multiple rounds of the trivia game.
41     """
42     if len(clients) < MAX_PLAYERS:
43         print("Not enough players to start a round.")
44         return
45     broadcast("Time will start in {WAIT_FOR_PLAYERS} seconds. Get ready!")
46     print(f"Waiting {WAIT_FOR_PLAYERS} seconds before starting the game...")
47     time.sleep(WAIT_FOR_PLAYERS)
48
49     while True:
50         # Enough questions exist to start a round
51         if len(question_db) < 3:
52             print("Not enough questions in the database to start a round.")
53             broadcast("Game cannot start due to insufficient questions. Please try later.")
54             return
55
56         # Select random questions from the database
57         round_questions = random.sample(list(question_db.items()), 3)
58         broadcast("The game is starting now!")
```

Figure 36: server code



```
59 # Select random questions from the database
60 round_questions = random.sample(list(question_db.items()), 3)
61 broadcast("The game is starting now!")
62
63 # Initialize answers for clients
64 for client in clients:
65     if client not in answers_per_client:
66         answers_per_client[client] = []
67
68 for i, (question, answer) in enumerate(round_questions, start=1):
69     time.sleep(QUESTION_DELAY)
70     broadcast(f"Question {i}: {question}")
71     print(f"Broadcasting Question {i}: {question}")
72     start_time = time.time()
73     client_answers = []
74
75     while time.time() - start_time < ANSWER_TIMEOUT:
76         try:
77             data, addr = server_socket.recvfrom(BUFFER_SIZE)
78             if addr in clients and addr not in client_answers:
79                 client_answers[addr] = data.decode().strip()
80                 print(f"({clients[addr]}) answered: {data.decode().strip()}")
81             except socket.timeout:
82                 continue
83
84     broadcast(f"Time's up! The correct answer was: {answer}")
85     print(f"Correct answer: {answer}")
86
87 # Evaluate answers
88 for client, response in client_answers.items():
89     if response.lower() == answer.lower():
```

Figure 37: server code

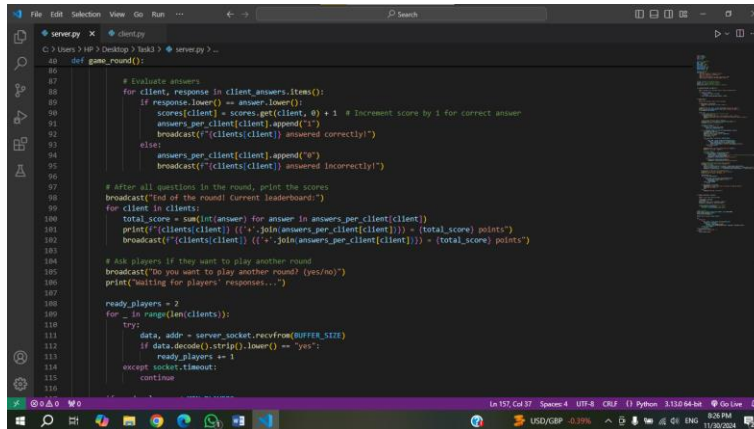


Figure 38: server code

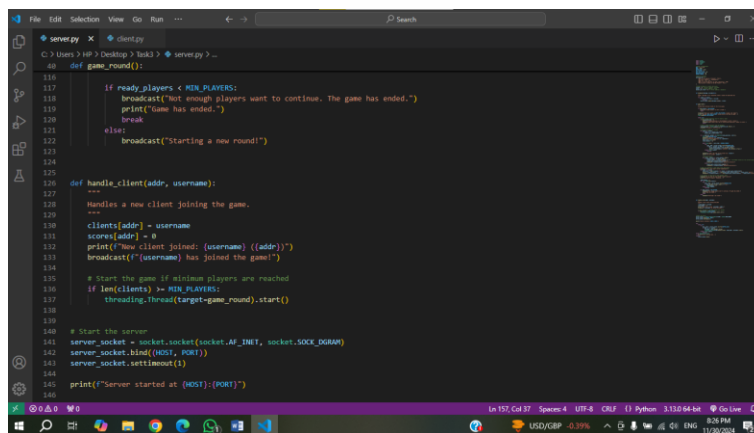


Figure 39: server code

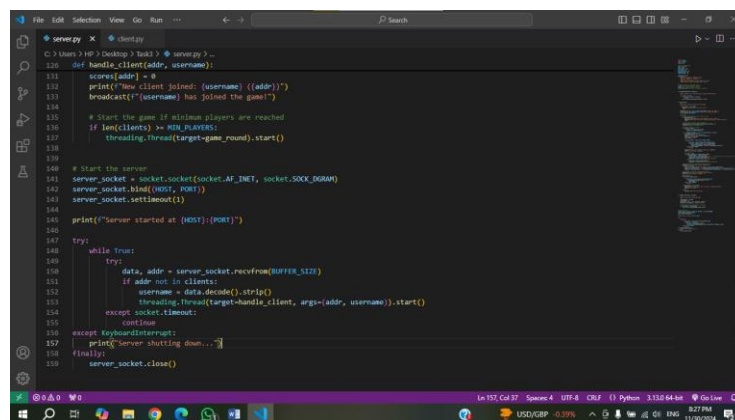
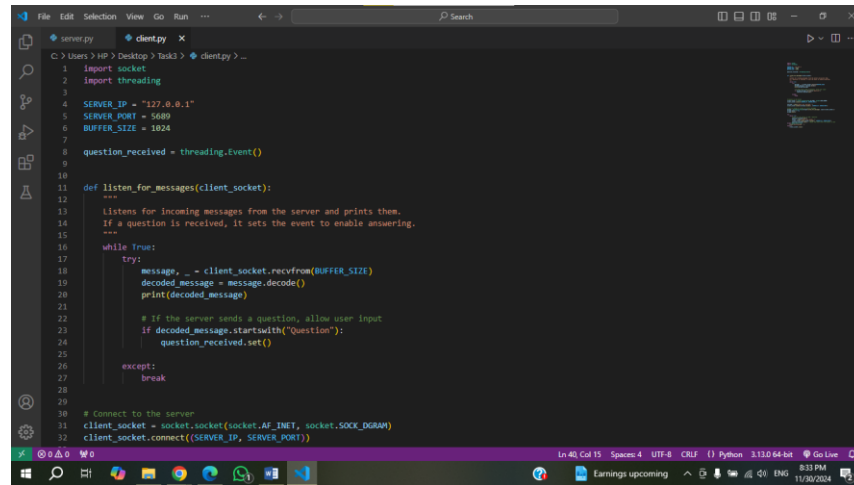


Figure 40: server code

- **Server code discussion:**

This Python code creates a multiplayer trivia game server using UDP. The server listens for players to join the game by connecting on port 5689, where each player is asked to provide a username. Once at least two players are connected, the server kicks off the game by randomly selecting three trivia questions from a pre-made list and sending them to all players. Each player has a set amount of time to answer, and the server collects their responses, checks if they're correct, and awards points. After each round, the server shows the current scores and asks if the players want to continue with another round. If not, enough players want to keep going, the game ends. The server uses threading to manage multiple players at once, ensuring the game runs smoothly for everyone.

- **The client code:**

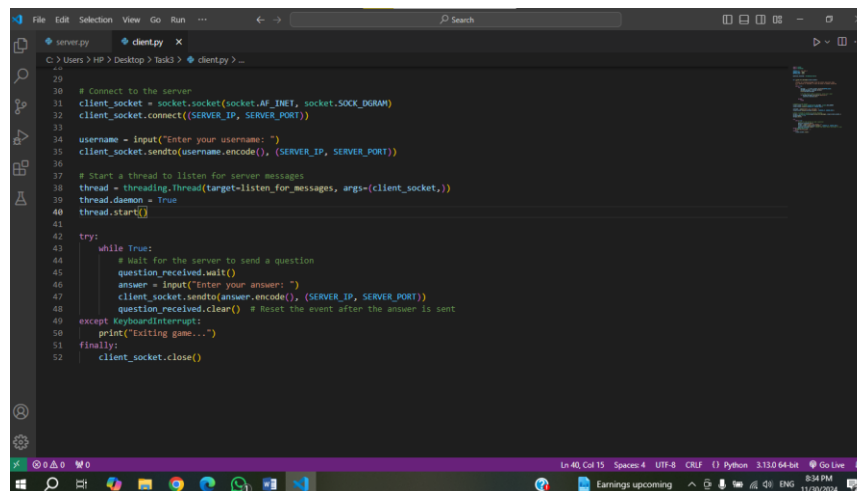


```

1  import socket
2  import threading
3
4  SERVER_IP = "127.0.0.1"
5  SERVER_PORT = 5689
6  BUFFER_SIZE = 1024
7
8  question_received = threading.Event()
9
10
11 def listen_for_messages(client_socket):
12     """
13     Listens for incoming messages from the server and prints them.
14     If a question is received, it sets the event to enable answering.
15     """
16     while True:
17         try:
18             message = client_socket.recvfrom(BUFFER_SIZE)
19             decoded_message = message.decode()
20             print(decoded_message)
21
22             # If the server sends a question, allow user input
23             if decoded_message.startswith("Question"):
24                 question_received.set()
25         except:
26             break
27
28 # Connect to the server
29 client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
30 client_socket.connect((SERVER_IP, SERVER_PORT))

```

Figure 41: the client code



```

29 # Connect to the server
30 client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
31 client_socket.connect((SERVER_IP, SERVER_PORT))
32
33 username = input("Enter your username: ")
34 client_socket.sendto(username.encode(), (SERVER_IP, SERVER_PORT))
35
36 # Start a thread to listen for server messages
37 thread = threading.Thread(target=listen_for_messages, args=(client_socket,))
38 thread.daemon = True
39 thread.start()
40
41 try:
42     while True:
43         # Wait for the server to send a question
44         question_received.wait()
45         answer = input("Enter your answer: ")
46         client_socket.sendto(answer.encode(), (SERVER_IP, SERVER_PORT))
47         question_received.clear() # Reset the event after the answer is sent
48     except KeyboardInterrupt:
49         print("Exiting game...")
50     finally:
51         client_socket.close()

```

Figure 42: the client code

- **Client code discussion:**

This Python code is for the client side of a trivia game, where the client connects to the server using UDP. After the client connects, it asks the user to enter a username, which is then sent to the server. A separate thread listens for messages from the server and prints them out. When the server sends a trivia question, it lets the user know that they can now answer by triggering an event. The client then waits for the question, prompts the user to type their answer, and sends the response back to the server. After each answer, the event is reset so the user can only answer one question at a time. The game continues until the user decides to exit.

- **The first example:**

```
File Edit Selection View Go ... task/network
# server.py # client.py
import socket
import sys
import random
import time

def game_round():
    # Generate a question
    print("Correct answer: ", answer)

    # Evaluate answers
    for client_response in client_answers.items():
        if response_lower == answer_lower:
            scores[client] + scores.get(client, 0) + 1 # Increment score by 1 for correct answer
            broadcast(f"{client[1]} answered correctly!")
            break
```

Microsoft Windows [Version 10.0.22631.4468]
© Microsoft Corporation. All rights reserved.
(C:\Users\qasra>cd C:\Users\qasra\Desktop\task\3network

C:\Users\qasra\Desktop\task\3network>python server.py
Server started at 0.0.0.0:5689
New client joined: hala ((127.0.0.1, 50603))
New client joined: lara ((127.0.0.1, 52213))
Waiting 20 seconds before starting the game...
Broadcasting Question 1: Which planet is known as the Red Planet?
lara answered: plato
Correct answer: Mars
hala: 0 points
lara: 0 points
Broadcasting Question 2: What is the capital of France?
hala answered: paris
lara answered: Paris
Correct answer: Paris
hala: 1 points
lara: 1 points
Broadcasting Question 3: What is the boiling point of water (celsius)?
Correct answer: 100
hala: 1 points
lara: 1 points

```
File Edit Selection View Go ... ↩ → tasknetwork
```

```
< client.py > server.py >
```

```
# client.py
```

```
from socket import *
import sys
import random

server_name = "localhost"
server_port = 8080

client_socket = socket(AF_INET, SOCK_STREAM)
client_socket.connect((server_name, server_port))

while True:
    # Receive message from server
    message = client_socket.recv(1024).decode()
    print("Received from server:", message)

    # Split the message into questions and answers
    questions_and_answers = message.split(',')

    # Iterate over each question and answer pair
    for i in range(len(questions_and_answers)):
        question = questions_and_answers[i]
        answer = questions_and_answers[i+1]

        # Ask the user for the answer
        user_answer = input(question + "? ")

        # Check if the user's answer is correct
        if user_answer.lower() == answer.lower():
            print("Correct answer!")
        else:
            print("Incorrect answer!")

        # Broadcast the correct answer to all clients
        broadcast_message = f"{question}, {answer}"
        broadcast_socket.send(broadcast_message.encode())

    # Wait for a short time before starting the next round
    time.sleep(5)
```

```
Command Prompt python .\tasknetwork
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\qasra> cd C:\Users\qasra\Desktop\tasknetwork
PS C:\Users\qasra\Desktop\tasknetwork> python client.py
Enter your username: hala
hala has joined the game!
lara has joined the game!
Game will start in 20 seconds. Get ready!
The game is starting now!
Question 1: Which planet is known as the Red Planet?
Enter your answer: mars
Time's up! The correct answer was: Mars
lara answered incorrectly!
Question 2: What is the capital of France?
Enter your answer: paris
Time's up! The correct answer was: Paris
lara answered correctly!
Question 3: What is the boiling point of water (Celsius)?
Enter your answer: 100
Time's up! The correct answer was: 100
End of the round! Current leaderboard:
```

The screenshot displays a Windows desktop environment. In the background, a web browser window shows the 'taskShark' application interface. The interface includes a sidebar with icons for 'client.py', 'server.py', and 'game round'. The main content area shows a list of client responses for a game round, including usernames, answers, and scores. The 'server.py' file is open in the background, showing code for handling client requests and broadcasting answers.

In the foreground, a Windows PowerShell terminal window is open, running a Python script named 'client.py'. The script is executing a game round, and the output shows the client's username, the game's question, and the client's answer. The script is also displaying the game's progress, including the time left and the current round's score.

```

C:\Users\qasra> cd C:\Users\qasra\Desktop\taskShark
C:\Users\qasra\Desktop\taskShark> python client.py
Enter your username: lara
lara has joined the game!
Game will start in 20 seconds. Get ready!
The game is starting now!
Question 1: Which planet is known as the Red Planet?
Enter your answer: pluto
Time's up! The correct answer was: Mars
lara answered incorrectly!
Question 2: What is the capital of France?
Enter your answer: Paris
Time's up! The correct answer was: Paris
lara answered correctly!
lara answered correctly!
Question 3: What is the boiling point of water (Celsius)?
Enter your answer: 21
Time's up! The correct answer was: 100
End of the round! Current leaderboard:
  
```

- The second example:

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\qasra> cd C:\Users\qasra\Desktop\task3network
PS C:\Users\qasra\Desktop\task3network> python server.py
Server started at 0.0.0.0:5689
New client joined: hala (('127.0.0.1', 64301))
New client joined: lara (('127.0.0.1', 53118))
Waiting 20 seconds before starting the game...
Broadcasting Question 1: What is 5 + 7?
hala answered: 12
lara answered: 17
Correct answer: 12
Broadcasting Question 2: What is the capital of France?
hala answered: Paris
lara answered: los angeles
Correct answer: Paris
Broadcasting Question 3: What is the boiling point of water (Celsius)?
Correct answer: 100
hala = 2 points
lara = 0 points
|

```

Figure 46: output of server EX2

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\qasra> cd C:\Users\qasra\Desktop\task3network
PS C:\Users\qasra\Desktop\task3network> python client.py
Enter your username: hala
hala has joined the game!
lara has joined the game!
Game will start in 20 seconds. Get ready!
The game is starting now!
Question 1: What is 5 + 7?
Enter your answer: 12
Time's up! The correct answer was: 12
hala answered correctly!
lara answered incorrectly!
Question 2: What is the capital of France?
Enter your answer: Paris
Time's up! The correct answer was: Paris
hala answered correctly!
lara answered incorrectly!
Question 3: What is the boiling point of water (Celsius)?
Enter your answer: 9
Time's up! The correct answer was: 100
End of the round! Current leaderboard:
hala = 2 points
lara = 0 points
|

```

Figure 47: output of client1 EX2

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\qasra> cd C:\Users\qasra\Desktop\task3network
PS C:\Users\qasra\Desktop\task3network> python client.py
Enter your username: lara
lara has joined the game!
Game will start in 20 seconds. Get ready!
The game is starting now!
Question 1: What is 5 + 7?
Enter your answer: 17
Time's up! The correct answer was: 12
hala answered correctly!
lara answered incorrectly!
Question 2: What is the capital of France?
Enter your answer: los angeles
Time's up! The correct answer was: Paris
hala answered correctly!
lara answered incorrectly!
Question 3: What is the boiling point of water (Celsius)?
Enter your answer: 6
Time's up! The correct answer was: 100
End of the round! Current leaderboard:
hala = 2 points
lara = 0 points
|

```

Figure 48: output of client2 EX2

- More than one round example:

```

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\qasra> cd C:\Users\qasra\Desktop\task3network
PS C:\Users\qasra\Desktop\task3network> python server.py
Server started at 0.0.0.0:5689
New client joined: hala ([('127.0.0.1', 56812)])
New client joined: lara ([('127.0.0.1', 57499)])
Waiting 20 seconds before starting the game...
Broadcasting Question 1: Who wrote 'Hamlet'?
Correct answer: Shakespeare
Broadcasting Question 2: Which planet is known as the Red Planet?
lara answered: ploto
Correct answer: Mars
Broadcasting Question 3: What is the capital of France?
hala answered: paris
lara answered: lo
Correct answer: Paris
hala (1) = 1 points
lara (0+0) = 0 points
Waiting for players' responses...
Broadcasting Question 1: Who wrote 'Hamlet'?
lara answered: shakespeare
Correct answer: Shakespeare
Broadcasting Question 2: What is the boiling point of water (Celsius)?
lara answered: 100
Correct answer: 100
Broadcasting Question 3: What is 5 + 7?
hala answered: 12
lara answered: 7
Correct answer: 12
hala (1+1) = 2 points
lara (0+0+0+1+0) = 1 points
Waiting for players' responses...
Broadcasting Question 1: Who wrote 'Hamlet'?
Correct answer: Shakespeare
Broadcasting Question 2: Which planet is known as the Red Planet?

```

Figure 49: output of server EX3

```

PS C:\Users\qasra\Desktop\task3network> python client.py
Enter your username: hala
hala has joined the game!
lara has joined the game!
Game will start in 20 seconds. Get ready!
The game is starting now!
Question 1: Who wrote 'Hamlet'?
Enter your answer: shakespeare! The correct answer was: Shakespeare
Question 2: Which planet is known as the Red Planet?
lara answered incorrectly!
Time's up! The correct answer was: Mars
lara answered incorrectly!
Question 3: What is the capital of France?
Enter your answer: paris
Time's up! The correct answer was: Paris
lara answered incorrectly!
lara answered incorrectly!
End of the round! Current leaderboard:
hala (1) = 1 points
lara (0+0) = 0 points
Do you want to play another round? (yes/no)
Starting a new round!
The game is starting now!
Question 1: Who wrote 'Hamlet'?
Enter your answer: me
Time's up! The correct answer was: Shakespeare
lara answered incorrectly!
Question 2: What is the boiling point of water (Celsius)?
Enter your answer: 4
Time's up! The correct answer was: 100
lara answered correctly!
Question 3: What is 5 + 7?
Enter your answer: 12
Time's up! The correct answer was: 12
hala answered correctly!
lara answered incorrectly!
End of the round! Current leaderboard:

```

Figure 50: output of client1 EX3

```

Starting a new round!
The game is starting now!
Question 1: What is 5 + 7?
Time's up! The correct answer was: 12
Question 2: Who wrote 'Hamlet'?
Time's up! The correct answer was: Shakespeare
Question 3: What is the capital of France?
Time's up! The correct answer was: Paris
End of the round! Current leaderboard:
hala (1+1+1+1) = 4 points
lara (0+0+0+1+0) = 1 points
Do you want to play another round? (yes/no)
Starting a new round!
The game is starting now!
Question 1: Who wrote 'Hamlet'?
Time's up! The correct answer was: Shakespeare
Question 2: Which planet is known as the Red Planet?
Time's up! The correct answer was: Mars
Question 3: What is 5 + 7?
Time's up! The correct answer was: 12
End of the round! Current leaderboard:
hala (1+1+1+1) = 4 points
lara (0+0+0+1+0) = 1 points
Do you want to play another round? (yes/no)
Starting a new round!
The game is starting now!
Question 1: Which planet is known as the Red Planet?
Time's up! The correct answer was: Mars
Question 2: What is the capital of France?
Time's up! The correct answer was: Paris
Question 3: What is the boiling point of water (Celsius)?
Time's up! The correct answer was: 100
End of the round! Current leaderboard:
hala (1+1+1+1) = 4 points
lara (0+0+0+1+0) = 1 points
Do you want to play another round? (yes/no)

```

Figure 51: output of client2 EX3

- Discussion:

When the server starts, it will let everyone know that it's up and running, waiting for players on port 5689. As players join, it will show their username and IP address. Once enough players are in, the server will announce that the game is starting and send out the first question. It will print the question for everyone to see, and as players respond, the server will display each answer, along with whether it was right or wrong. At the end of each round, the server will share the scores, showing how each player did. If players want to continue, it will ask if they're ready for another round, but if not, enough players are interested, the game will end. When the server shuts down, it will let everyone know it's closing. On the client side, when a player connects, they'll see a message confirming they're connected to the server. As the game progresses, the client will display messages about the game starting, the questions being asked, and updates to the leaderboard. The client will prompt the player for an answer when a question arrives, and once the answer is submitted, it will confirm that the answer was sent. At the end of the round, the client will show the updated scores and any disconnection messages if the game is over or the server shuts down.

Conclusion

To conclude, this project provides an understanding of networking concepts, focusing on socket programming and real-time communication. By building a simple web server and a multiplayer trivia game, we've gained practical experience with TCP and UDP sockets, learning how to manage client-server interactions. Additionally, using network tools like ping, tracer, and Wireshark has helped us improve our ability to analyze and troubleshoot network traffic. Overall, this project has not only strengthened our technical skills in programming, web development, and networking but also taught us the value of collaboration and teamwork.

Teamwork

For our project, we formed a team of three students to make sure everyone had an equal share of the work. We were open meetings and worked together on every task of the project, from writing the codes until the results shown. By working closely together, we made sure to do the project completely.

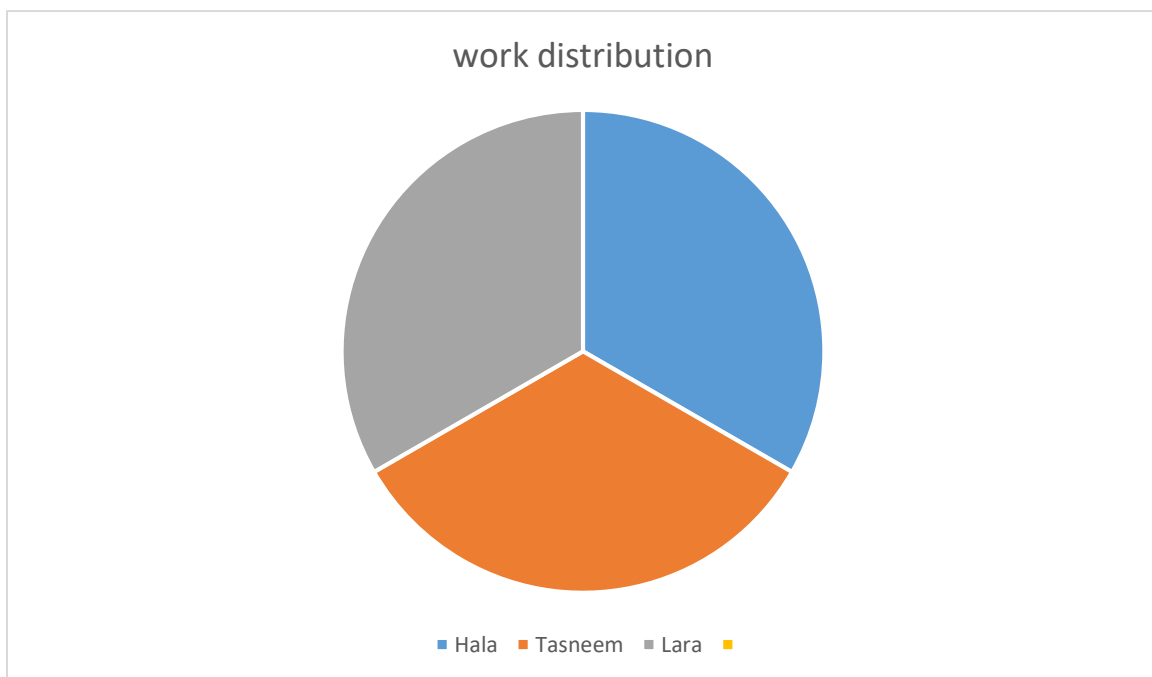


Figure 52: Work distribution

References

- https://gaia.cs.umass.edu/kurose_ross/index.php
- https://www.youtube.com/watch?v=zYYJWzK1_p8&list=PL7YSA38ZxcXzG529R9e2ahWzp6YYMkJmT&index=32
- [Examples \(Dr. Ibrahim\) - Google Drive](#)